

Wood Depot

Contents

3	Project Abstract.....	2
3.1	Sprint Plan (Scrum Methodology)	3
3.2	Non-Functional Requirements (Extended).....	7
3.3	Technology Stack & Justification	8
3.4	UML Component Diagram & Architecture Description.....	10
3.5	Testing Strategy	12
3.6	Glossary (Expanded)	13
3.7	APA References	14

3 Project Abstract

The rapid growth of e-commerce has revolutionized how shoppers interact with markets, especially in categories where convenience, the ability to customize their items and variety are paramount to the consumer. Wood Depot is a new concept and online furniture store that provides customers with a creative shopping experience, offering an e-commerce environment that connects buyers with interior designers and sellers. Unlike traditional shopping systems that limit consumers in terms of stock available or store open hours, Wood Depot is an online marketplace that is open 24/7 and has a range of furniture categories for example: beds, sofas, side tables, wardrobe, study tables, kitchen furniture and décor products. The focus at Wood Depot is both on ready-made and custom-made furniture allowing shoppers to design their living spaces in a manner suited to their tastes while providing interior designers with the opportunity to showcase their portfolios, market their furniture products and receive feedback from the shopper in order to enhance service offerings.

The system framework was created using PHP for front-end capabilities and MySQL for database management and support with technologies like HTML5, CSS3, JavaScript, AJAX, and jQuery to provide the most interactive, responsive, and user-friendly UI. Multi-role management allows the site administrator (super admin), to delineate priorities and responsibilities between the site's super admin, sub-admins, customers, interior designers, and visitors. This provides specific roles for all actors related to an online marketplace, such as order processing, product management, inquiry management, payment tracking, and customer feedback management. This approach provides patrons with quick checkout, logs purchases as a secure transaction, develops an infrastructure for designers and sellers motivated for growth through list sales and purchase points collection, and is viable for i) businesses selling products, ii) businesses supporting development awareness, iii) and providing customers living in recreation-oriented communities an orientation to retail product reliance.

The project combines strong database structures, defined role management and a simple user interface to address the restrictions of traditional retail furniture

shopping that include limited availability, a significant time investment, and personalized recommendations. Furthermore, the feedback and rating system allows customers to make educated purchasing decisions while allowing designers a valuable feedback tool to improve their services. Wood Depot is ultimately intended to be a full-service online furniture marketplace incorporating convenience, affordability, and creativity while providing a basis for the next evolution of digital commerce in the home décor and furniture market.

3.1 Sprint Plan (Scrum Methodology)

Sprint 0 (Week 0 – Preparation Phase)

- **Duration:** 1 week (Pre-Sprint setup)
- **Goal:** Project environment setup and backlog preparation.
- **Activities:**
 - Finalize product backlog with prioritized features.
 - Define roles (Product Owner, Scrum Master, Development Team).
 - Set up version control, project management tools (e.g., Jira/Trello), and development environment (XAMPP, PHP, MySQL).
 - Conduct Sprint Planning Meeting.

Sprint 1 (Week 1–2)

- **Sprint Goal:** Establish basic system foundation and user management.
- **User Stories:**
 - As a **visitor**, I want to register/login so that I can access the platform.

- As an **admin**, I want to manage user accounts so that I can control access.
- **Tasks:**
 - Implement user registration, login, and authentication system.
 - Create admin dashboard (basic version).
 - Develop database tables for users (customers, designers, admins).
 - Unit testing for authentication & session management.
- **Deliverables:**
 - Working login/registration module.
 - Functional admin dashboard (basic version).

Sprint 2 (Week 3–4)

- **Sprint Goal:** Product catalog and category management.
- **User Stories:**
 - As a **customer**, I want to view products by category so I can browse easily.
 - As an **admin**, I want to add/manage product categories and products.
- **Tasks:**
 - Develop product catalog (categories & subcategories).
 - Implement product CRUD operations for admin.
 - Create product detail page (with images & description).
 - Integrate basic cart functionality.

- **Deliverables:**
 - Fully functional product catalog with categories.
 - Admin product management features.

Sprint 3 (Week 5–6)

- **Sprint Goal:** Cart, orders, and payment integration.
- **User Stories:**
 - As a **customer**, I want to add items to my cart and place an order.
 - As an **admin**, I want to process orders and manage order statuses.
- **Tasks:**
 - Implement shopping cart system (add, update, remove items).
 - Order placement workflow (checkout process).
 - Payment gateway simulation (basic integration).
 - Order management for admin (processing, shipped, delivered).
- **Deliverables:**
 - End-to-end order placement & processing flow.
 - Admin order tracking system.

Sprint 4 (Week 7–8)

- **Sprint Goal:** Feedback system, designer module, and final refinements.
- **User Stories:**

- As a **customer**, I want to give feedback & ratings on purchased products.
- As a **designer**, I want to showcase my projects and interact with customers.
- As a **visitor**, I want to browse designer portfolios before registering.

- **Tasks:**

- Implement feedback & rating system for products and designers.
- Designer module (profile, portfolio/project upload, inquiry response).
- Newsletter & CMS page management (About Us, Contact).
- Final UI improvements, bug fixing, system testing.

- **Deliverables:**

- Complete customer feedback system.
- Interior designer portfolio & inquiry management.
- Deployment-ready system.

3.2 Non-Functional Requirements (Extended)

Attribute	Specification
Performance	Complete all page loads (home page, product catalog, cart) in less than 3 seconds, for a normal 4G connection; API response times for catalog queries with fewer than 200 products need to come back within 500ms.
Scalability	The application must be able to support 500 concurrent users at a minimum, and horizontal scalability must be available for increased number of users and products.
Usability	New customers should be able to move through UI, search for products, add to cart within one minute of first landing; designers should be able to upload their portfolio without needing training.
Security	In order to prevent SQL injection, and XSS substitution, any inputs must be sanitized, passwords should be stored using hashing, use JWT's to do multi-user authentication, and that HTTPS/SSL is required.
Maintainability	The code base should contain modular designs while separating areas of concern (MVC pattern in PHP); the downtime to upgrade should be no more than < 2 hours.
Portability	The system must be compatible on Windows and Linux servers with PHP ≥ 7.4 and MySQL ≥ 5.7 ; supported browsers: Chrome, Firefox, Edge.
Data Integrity	All transactions (or orders, payments) must be ACID-compliant; specifically, we must not allow duplicate or lost orders even if load concurrently occurs.
Accessibility	The UI must meet WCAG 2.1 Level AA standards for accessibility for users with visual or motor impairments.
Backup & Recovery	Automated backups must occur daily, and the system must have a restore functionality that will restore the system to the last backup restore point within 1 hour of a major failure.

3.3 Technology Stack & Justification

Front-End

- **Technologies: HTML5, CSS3, JavaScript, jQuery, Bootstrap**
- **Justification:**

The front-end has a standard tech stack for website layout and styling: HTML5 and CSS3; interactive and dynamic content are done with JavaScript and jQuery; and the site design uses Bootstrap to be responsive to mobile devices. This stack is simple, has a large community support base, and is easy to link with back-end PHP.

Back-End

- **Technology: PHP (Core PHP with MVC approach)**
- **Justification:**

PHP is a popular open-source scripting language for developing web-based applications. PHP can interface with databases such as MySQL and supports prototyping quickly enough to be functional in an iterative approach utilizing Scrum development. The use of an MVC structure adds maintainability. As our team adds new features to our project, it is helpful to modularize the code to aid with understanding how to scale the project in the future.

Database

- **Technology: MySQL (MySQLi)**
- **Justification:**

MySQL is a strong relational database management system (RDBMS) that supports ACID compliance, is scalable, and allows secure transaction processing. The fact that MySQL interacts with PHP and has been widely adopted makes it an attractive solution. MySQLi (improved extension) offers

more reliable security, performance, support for prepared statements for all queries, and most importantly protects against SQL injections.

Web Server

- **Technology: XAMPP Server (Apache, MySQL, PHP, Perl)**

- **Justification:**

XAMPP freely puts together a very convenient local development environment with Apache (web server), PHP interpreter, and MySQL. This convenience affords us a smooth testing and debugging experience during the development phase while also ensuring we can maintain an environment that is consistent with that of a production server.

Security Layer

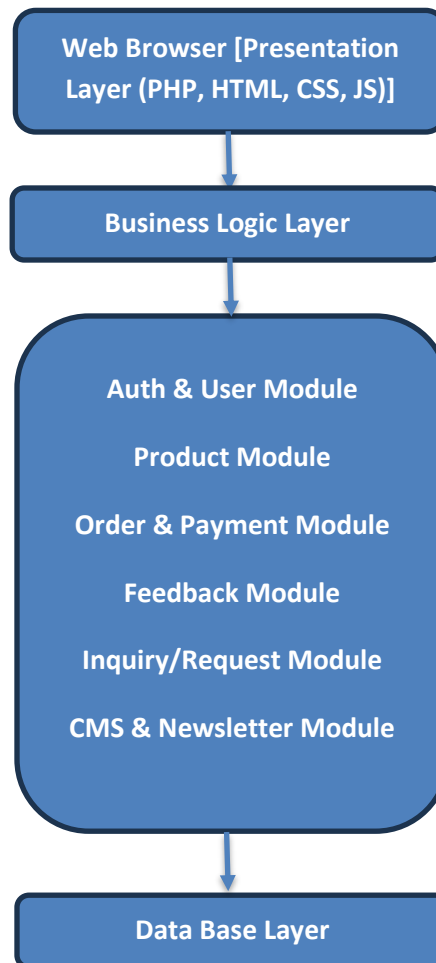
- **Technologies: JWT (JSON Web Tokens), SSL/TLS, Input Sanitization Libraries**

- **Justification:**

JWT will provide secure session management for multiple users' roles (i.e. admin, customer, designer). SSL/TLS will encrypt communication between the client and server. Input sanitizing libraries will mitigate common attack vectors such as XSS (Cross Site Scripting) and SQL Injection, which will give a secure and trusted platform for the user.

3.4 UML Component Diagram & Architecture Description

The system consists of the following components:



➤ Architecture Description

1. Architecture Style:

- Three-tier architecture
 - Presentation Layer: HTML5, CSS3, JS, JQuery, AJAX (Client-Side).
 - Application Layer: PHP (Server-side logic).
 - Data Layer: MySQL (persistent storage).

2. Key Components:

- User Interface: Responsive web pages for Admin, Designer, Customer, Visitor.
- Application Services: Business logic for authentication, order processing, payments, content management, etc.
- Database Services: Data storage for users, products, orders, payments, inquiries, and feedback.

3. Interactions:

- Customers interact with the system via the web browser.
- Interior Designers manage portfolios and respond to requests.
- Admins oversee platform operations, manage categories, users, payments, and content.
- Database handles persistent data storage with foreign key relations ensuring integrity.

4. Deployment:

- Client Side: Any device with a modern browser.
- Server Side: Runs on XAMPP (Apache + PHP + MySQL).
- Database: MySQL, connected through PHP (MySQLi).

All components are loosely coupled and communicate via HTTP APIs.

3.5 Testing Strategy

Module	Test Type	Tool	Details
Authentication (Login, Registration)	Unit + Integration	PHPUnit / Pytest (if backend in Python)	Validate login with valid/invalid credentials, session handling, password reset.
Product Management	Unit + Integration	PHPUnit + MySQL Test DB	Test add/update/delete product, category mapping, image upload validation.
Cart & Order Module	Integration + E2E	Postman + PHPUnit	Verify cart operations, order placement, payment flow, order status updates.
Payment Gateway	Integration + Mock Test	Postman / Mock Server	Test successful, failed, and canceled payments. Validate DB transaction entries.
Feedback & Rating	Unit Test + UI Test	PHPUnit + Browser	Submit feedback with rating, validate stored values, check UI display.
System Workflow (E2E)	End-to-End Test	Postman + Selenium	Register → Add to Cart → Place Order → Payment → Feedback flow validation.

Frontend UI	Manual Test + Automated UI	Browser (Chrome/Firefox) + Selenium	Check form validation, navigation flow, error messages, responsiveness.
Performance & Load	Stress/Load Test	JMeter / Locust	Simulate 500–1000 concurrent users browsing and placing orders.

3.6 Glossary (Expanded)

Term	Definition
AST (Abstract Syntax Tree)	A tree-structured representation of code syntax, often used in compilers and code analyzers to understand the structure of source code.
Transpiler	A source-to-source compiler that converts code from one programming language into another at the same abstraction level (e.g., TypeScript → JavaScript).
REST API	Representational State Transfer Application Programming Interface: a standardized web interface for communication between client and server using HTTP methods such as GET, POST, PUT, DELETE.
Tokenization	The process of breaking a string of text or code into smaller units called tokens (keywords, identifiers, operators, etc.), which are then used for parsing and analysis.
Code Converter	A tool or program that automatically translates code written in one programming language into another, preserving logic while adapting syntax.

3.7 APA References

PHP Group. (n.d.). *PHP: Hypertext Preprocessor*. <https://www.php.net/>

W3Schools. (n.d.). *PHP tutorial*. W3Schools Online Web Tutorials.
<https://www.w3schools.com/php/>

W3Schools. (n.d.). *CSS tutorial*. W3Schools Online Web Tutorials.
<https://www.w3schools.com/css/>

Coyier, C. (n.d.). *CSS-Tricks*. CSS-Tricks. <https://css-tricks.com/>

Bootstrap Team. (n.d.). *Bootstrap documentation*. <https://getbootstrap.com/>

W3Schools. (n.d.). *JavaScript tutorial*. W3Schools Online Web Tutorials.
<https://www.w3schools.com/js/>

GitHub Link:- <https://github.com/Mayank-Dobariya/Wood-Depot>